

3D Import Inspection Implementation Prompt

A detailed coding-agent prompt for visualizing probable railway flaws from uploaded CSV and JSON data while preserving human approval.

Copy the prompt below into your coding agent. It is tailored to the current RailMind AI repository and its existing React Three Fiber, Flask, Zustand, and import/export architecture.

Core principle: the 3D diagram is an evidence-based inspection aid, not proof that the defect is confirmed.

Implementation Prompt

You are a senior full-stack engineer working on the RailMind AI railway operations platform. Implement an import-to-3D-inspection workflow for uploaded CSV and JSON railway event files.

Project Context

The repository contains Flask and SQLAlchemy on the backend, React and TypeScript with Vite on the frontend, Three.js with React Three Fiber for 3D, and Zustand for state. Existing relevant files include:

- frontend/src/pages/ImportExport.tsx
- frontend/src/stores/useImportStore.ts
- frontend/src/3d/AssetVisualizer3D.tsx
- frontend/src/config/api.ts
- backend/app/services/import_service.py
- backend/app/api/routes/imports.py

The existing import flow supports CSV/JSON upload, validation, event summaries, AI next-step suggestions, import confirmation, exports, and a basic 3D asset visualizer.

Main Feature

When a file contains a track defect, signal failure, overhead equipment failure, bridge risk, train issue, or station issue, parse the event, identify the affected asset, generate a 3D visualization payload, display the probable flaw, show evidence and confidence, suggest a next action, require explicit approval, and record the decision. Do not automatically execute operational actions.

Data Model Changes

Add these optional fields while keeping existing files compatible:

```
asset_id
asset_type
track_id
station_id
train_id
chainage_m
latitude
longitude
defect_type
defect_position
confidence
sensor_source
inspection_timestamp
evidence
```

Example CSV

```
event_type,severity,description,affected_passengers,estimated_delay_minutes,priority,station_id,track_id,chainage_m,defect_type,conf
track_defect,critical,Possible rail fracture near Howrah Junction,2500,120,9,1,12,4380,rail_fracture,0.86,track_inspection_vehicle
signal_failure,high,Signal failure affecting eastbound services,800,45,8,1,7,4210,signal_failure,0.78,signal_monitoring_system
```

Backend Requirements

Modify `backend/app/services/import_service.py` and `backend/app/api/routes/imports.py`.

1. Normalize optional fields safely. Update `_normalize_event()` to parse asset IDs, asset types, chainage, coordinates, defect type, confidence, source, and timestamp. Malformed optional values must not crash the parser.
2. Add `build_visualization_payload(event)`. For track events, return an asset type, asset ID, defect type, deterministic 3D position, confidence, severity, label, and evidence. Use `chainage_m` for a demo position when GIS coordinates are unavailable, and mark it as estimated.

```
{
  "asset_type": "track",
  "asset_id": "TRK-12",
  "defect": {
    "type": "rail_fracture",
    "position": {"x": 4.38, "y": 0.25, "z": 0},
    "confidence": 0.86,
    "severity": "critical",
    "label": "Probable rail fracture"
  },
  "evidence": {
    "description": "Possible rail fracture near Howrah Junction",
    "chainage_m": 4380,
    "source": "track_inspection_vehicle",
    "position_accuracy": "estimated"
  }
}
```

3. Include visualizations in the preview response, with one visualization per visualizable event.
4. Generate an `open_3d_inspection` suggestion with an asset ID, visualization ID, confidence, and `auto_triggered=false`.
5. Prevent duplicate agent actions. Treat actions already triggered during import as already processed, or require an explicit rerun confirmation.

Frontend Requirements

Modify the import store and import page. You may create `frontend/src/components/import-export/ImportedDefectViewer.tsx`.

Add a typed `DefectVisualization` interface containing the asset type, asset ID, defect position, label, confidence, severity, estimated-position flag, and evidence fields. Add suggestion decision states: pending, approved, rejected, executed, failed, and already_processed.

Create a React Three Fiber viewer with reusable procedural models:

- Track: two rails, sleepers, ballast, labels, and defect marker.
- Signal: mast, light, and failure marker.
- Train: carriages and delay marker.
- Platform: platform slab and crowd-density markers.
- Generic: simple asset outline and marker.

Use `OrbitControls`, a reset-camera control, responsive sizing, deterministic positions, and only subtle marker or camera animation. Use red/orange/yellow/blue markers for critical/high/medium/low severity. Labels must say probable, suspected, or estimated unless the data proves the defect.

Integration and User Review

On the import preview step, show the 3D viewer when visualizations exist. Provide an incident selector for multiple assets, an evidence panel, a confidence indicator, and Approve/Reject controls. Approving `open_3d_inspection` selects and focuses the visualization; it must not close tracks, dispatch teams, or modify operations. Rejection updates the card without executing anything.

Reuse existing export and navigation handlers. Inspect `frontend/src/config/api.ts` because the response interceptor already returns `response.data`; avoid incorrectly reading `res.data` from already-unwrapped responses.

Demo Scenario

Support this JSON upload and make the first incident render as a track with a red probable-fracture marker:

```
[
  {
    "event_type": "track_defect",
    "severity": "critical",
    "description": "Possible rail fracture near Howrah Junction",
    "station_id": 1,
    "track_id": 12,
    "chainage_m": 4380,
    "defect_type": "rail_fracture",
    "confidence": 0.86,
    "sensor_source": "track_inspection_vehicle",
    "affected_passengers": 2500,
    "estimated_delay_minutes": 120,
    "priority": 9
  },
  {
    "event_type": "signal_failure",
    "severity": "high",
    "description": "Signal failure affecting eastbound services",
    "station_id": 1,
    "track_id": 7,
    "chainage_m": 4210,
    "defect_type": "signal_failure",
    "confidence": 0.78,
    "sensor_source": "signal_monitoring_system"
  }
]
```

Expected behavior

- Upload JSON and see two visualizable incidents.
- See a track model and an 86% confidence probable rail-fracture marker.
- Review the evidence and suggested inspection action.
- Approve or reject the suggestion without automatic operational changes.
- Keep existing import, export, and agent behavior working.

Validation Requirements

Run these commands after implementation:

```
cd frontend
npm run type-check
npm run build
npm run lint

cd backend
python -m compileall app
```

Also verify old files without new fields, malformed optional values, deterministic rendering, estimated-coordinate fallback, duplicate-agent prevention, working exports, responsive layout, and a nonblank fallback viewer.

Keep changes focused. Reuse the existing 3D system and do not rewrite unrelated pages.